

Engenharia de Software Aplicada

Desenho detalhado – Sprint nºXX - Nome da Sprint

Turma: ESA PL1
Grupo nº: OP
nº202200037 Alexandre Miguel

Versões do Trabalho

Versão	Data	Autor	Descrição
1	30/06/2026	Alexandre Miguel	Documento da sprint
2	08/07/2026	Alexandre Miguel	Documento oficial

			completo sobre template

1	SUMÁRIO EXECUTIVO	4
2	INTRODUÇÃO	4
3	DESENHO DETALHADO	5
3.1	Introdução	5
3.2	Módulo(s) M07 / Sprint 5 - DevOps, Administração e Entrega Final	5
3.2.1	Requisitos funcionais implementados	5
3.2.2	Diagrama de classes da Sprint 5	6
3.2.3	Processos de negócio da Sprint 5	7
3.2.4	Análise de estados relevantes da Sprint 5	7
3.2.5	Interface com o utilizador da Sprint 5	7
3.3	Testes	8
3.3.1	Testes Unitários	8
3.3.2	Testes de Automação	8
3.3.3	Testes de Integração	8
3.3.4	Testes de Sistema	8
3.3.5	Testes de aceitação	9
3.4	Manual de utilização	9
3.5	Manual técnico	9

SUMARIO EXECUTIVO

A Sprint 5 teve como objetivo Docker, Azure, backoffice, i18n, tema, métricas e documentação. Este documento detalha requisitos implementados, classes, processos de negócio, interface, testes, manual de utilização e manual técnico.

1 INTRODUÇÃO

O desenho detalhado descreve como os requisitos da sprint foram implementados e validados na aplicação ASP.NET Core MVC.

O incremento da Sprint 5 foi planeado a partir do backlog Jira e validado por execução local/Docker, seguindo os módulos M07.

2 DESENHO DETALHADO

2.1 Introdução

A Sprint 5 foi detalhada a partir dos requisitos selecionados no backlog, das decisões de desenho tomadas no desenho de alto nível e das evidências implementadas no código ASP.NET Core MVC.

2.2 Módulo(s) M07 / Sprint 5 - DevOps, Administração e Entrega Final

2.2.1 Requisitos funcionais (implementados)

ID	Módulo	Prioridade	Estado	Requisito
RF-M07-01	M07	Must	Implementado	O sistema deverá disponibilizar backoffice para estado de integrações e utilizadores.
RF-M07-02	M07	Must	Implementado	O sistema deverá disponibilizar endpoint de saúde operacional.
RF-M07-03	M07	Should	Implementado	O sistema deverá disponibilizar modo claro/escuro e português/inglês.
RF-M07-04	M07	Must	Implementado	O sistema deverá executar em Docker com base de dados persistente.

Os requisitos abaixo foram considerados no incremento e encontram-se implementados ou operacionalmente suportados na aplicação.

2.2.2 Diagrama de classes de desenho detalhado da Sprint 5

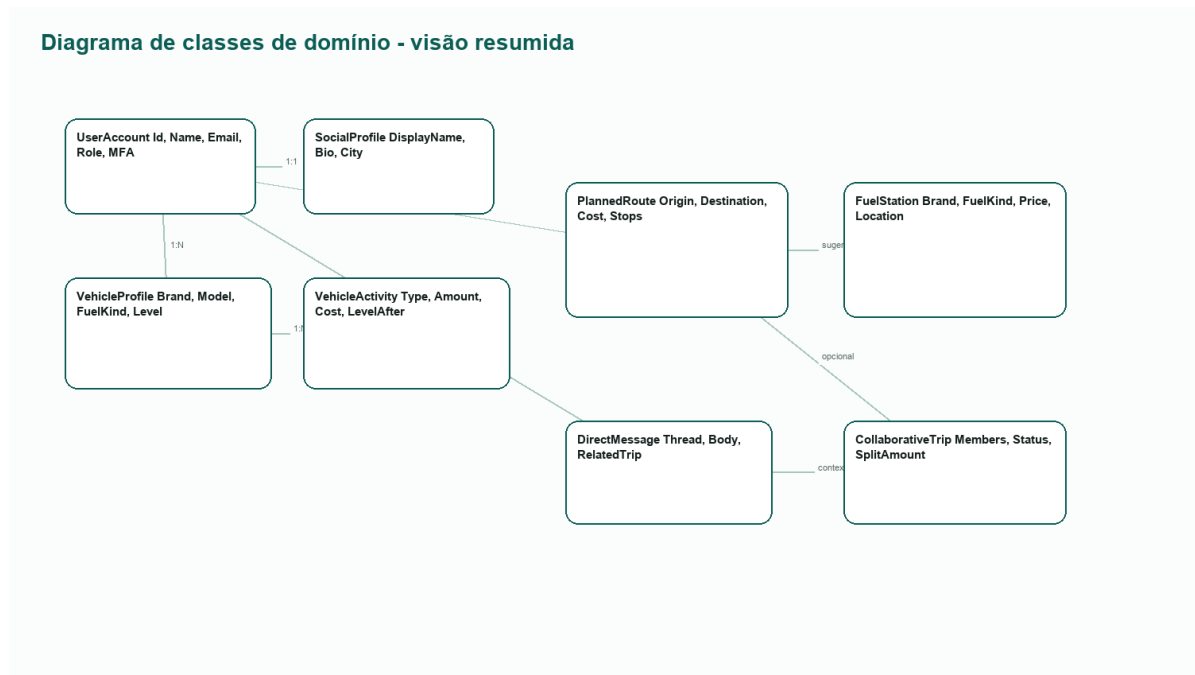


Figura 1 - Diagrama de classes de desenho detalhado.

O diagrama representa as entidades persistidas e serviços relevantes para o incremento.

2.2.3 Diagramas de processos de negócio da Sprint 5

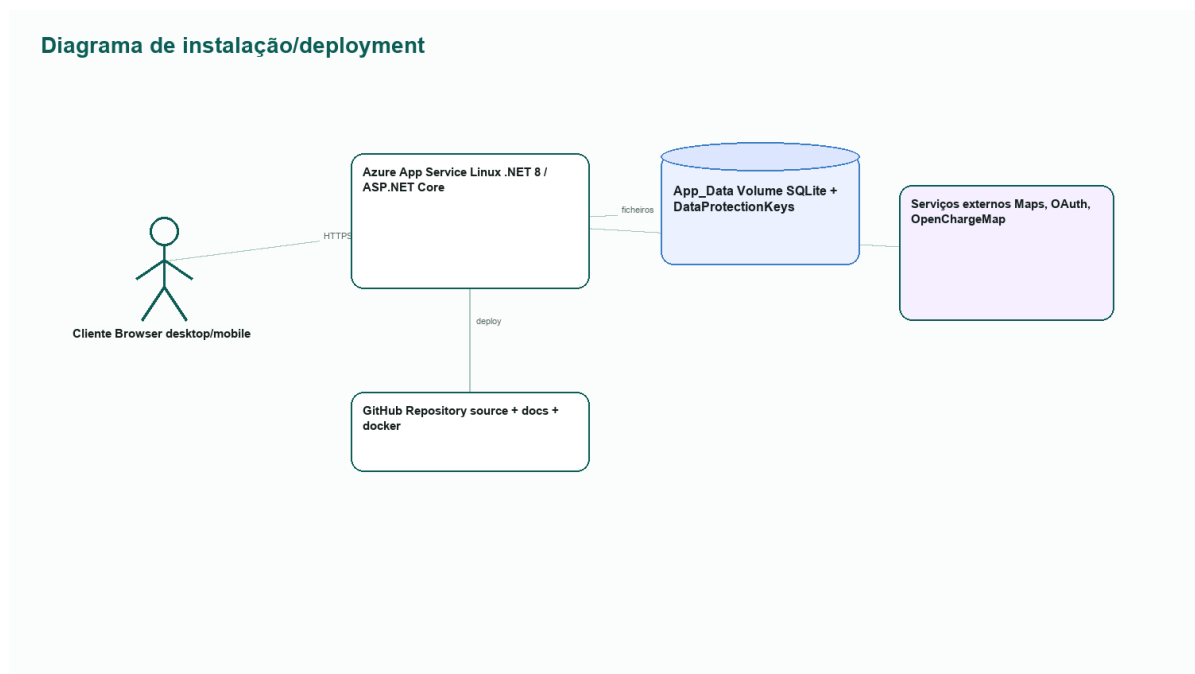


Figura 2 - BPD da Sprint 5.

O processo de negócio é representado em BPD/BPMN, usando lanes e fluxos compatíveis com a notação trabalhada na unidade curricular.

2.2.4 Análise de estados relevantes da Sprint 5

2.2.5 Interface com o utilizador da Sprint 5

A interface foi validada diretamente nas views Razor correspondentes, mantendo navegação clara entre as áreas do sistema.

Diagrama de Navegação

Origem	Destino	Condição
Login	Perfil	Autenticação válida
Perfil	Garagem	Utilizador autenticado
Garagem	Planeamento	Veículo selecionado
Planeamento	Garagem	Rota aplicada
Perfil	Social	Perfil social ativo

Ecrãs e fluxos de utilização implementados

2.3 Testes

Os testes combinaram validação unitária, integração com serviços, execução manual orientada a cenário e aceitação funcional.

3.3.1 Testes Unitários

2.3.1.1 Especificação dos Casos de testes:

Nome Caso de teste:	
Código:	
Finalidade:	
Entradas:	
Resultados esperados:	
Dependências:	

2.3.1.2 Especificação dos Procedimentos:

Nome Caso de teste:	
Código:	
Preparação:	
Inicialização:	
Recursos específicos:	

2.3.1.3 Resultados:

Nome Caso de teste:	
Código:	
Responsável:	
Período de teste:	

Resultados obtidos:	
Observações:	

2.3.2 Testes de Automação

Os testes automatizados podem ser executados por dotnet test e incidem sobretudo sobre serviços de domínio e estado aplicacional.

2.3.3 Testes de Integração

A estratégia adotada foi incremental, validando integrações com base de dados, APIs externas configuráveis e fluxos MVC.

2.3.3.1 Testes de Regressão

2.3.3.2 Testes de Integração com 3rdparty

2.3.4 Testes de Sistema

Os testes de sistema validam adequação funcional, compatibilidade, segurança, eficiência e capacidade de interação.

2.3.4.1 Testes de Funcionalidade

2.3.4.2 Testes de Eficiência (Carga)

2.3.4.3 Compatibilidade

2.3.4.4 Capacidade de interação

2.3.4.5 Testes de Confiabilidade

2.3.4.6 Segurança

2.3.4.7 Testes de Manutabilidade

2.3.4.8 Testes de Flexibilidade

2.3.4.9 Testes de Segurança Física (Safety)

2.3.5 Testes de aceitação

A aceitação baseia-se na demonstração do fluxo funcional previsto para a sprint e na rastreabilidade entre requisito, implementação e evidência.

2.4 Manual de utilização

O utilizador inicia sessão, acede ao módulo correspondente, preenche os dados obrigatórios, valida o resultado e guarda/aplica as alterações quando necessário.

O manual de utilização é orientado ao percurso real do utilizador na aplicação.

A ajuda ao utilizador está distribuída pelas próprias páginas, com validação de campos e mensagens de erro contextuais.

2.5 Manual técnico

Executar dotnet build, dotnet test e docker compose. A configuração externa é feita por variáveis de ambiente e ficheiro .env local.

A documentação técnica é mantida no Confluence e complementada por README, scripts Docker e configuração por variáveis de ambiente.